



REINFORCEMENT LEARNING

ANAUM HAMID

What is Reinforcement Learning?

An agent learns to make decisions by interacting with an environment, taking actions, and receiving rewards or penalties — with no labeled data needed.

Trial & Error → Learning

Agent

Learner / Decision Maker

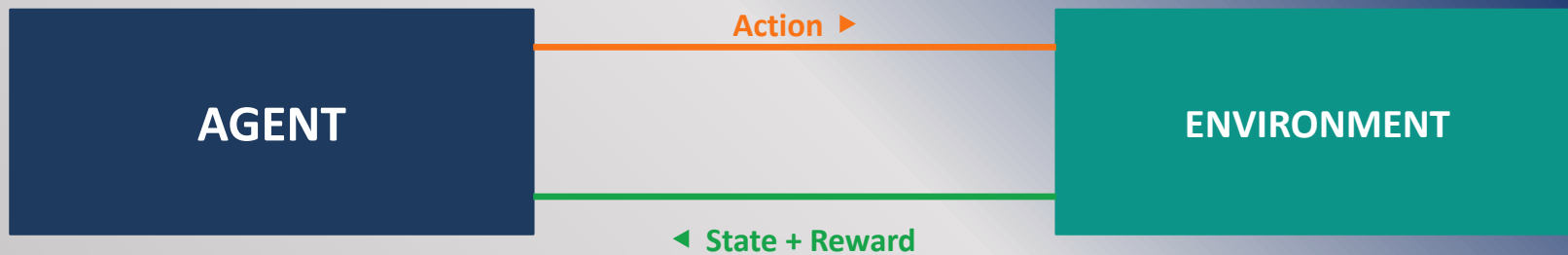
Environment

World the agent acts in

Reward

Feedback signal (+/-)

The RL Process

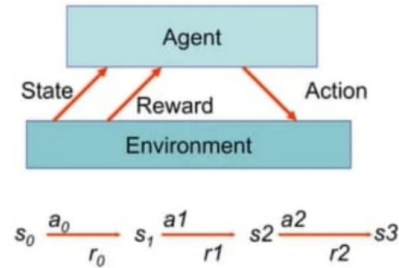


S	A	R	π	V
State	Action	Reward	Policy	Value
Current situation of the agent in the environment	Choice made by the agent at each step from set of Actions	Immediate feedback signal (+good / -bad) against last action	Strategy, (mapping states → actions), to decide the next action based on the current State	Expected long-term reward from a state with discount, opposite to R.

- Each percept(e) is enough to determine the State(the state is accessible)
- **Agent's task:** Find a optimal policy by mapping states of environment to actions of the agent, that maximize long-run measure of the reward (reinforcement)
- It can be modeled as Markov Decision Process (MDP) model.
 - Markov decision process (**MDP**) is a a mathematical framework for **modeling** decision making i.e mapping a solution in reinforcement learning.

MDP model

- MDP model $\langle S, T, A, R \rangle$

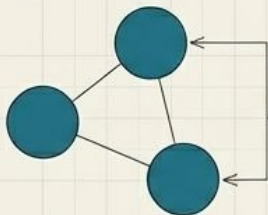


- S — set of states
- A — set of actions
- **Transition Function:** $T(s, a, s') = P(s'|s, a)$ – the probability of transition from s to s' given action a
 $T(s, a) \rightarrow s'$
- **Reward Function:** $r(s, a) \rightarrow r$ the expected reward for taking action a in state s

$$R(s, a) = \sum_{s'} P(s'|s, a) r(s, a, s')$$

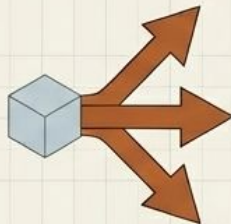
$$R(s, a) = \sum_{s'} T(s, a, s') r(s, a, s')$$

The mathematical reality of the agent is defined by the Markov Decision Process.



States (S)

The set of all possible situations.



Actions (A)

The set of all available moves from a state.

Transitions $T(s, a, s')$

Probability of landing in a new state given an action. The Markov property guarantees the future depends only on the current state, not the past.



Rewards $R(s, a, s')$

The immediate feedback.

The ultimate goal of an MDP is to find the Optimal Policy (π^*) — a universal mapping from every State to the best possible Action to maximize long-term, discounted utility.

Types of Reinforcement Learning

How knowledge is built

When/how the policy is updated



Model-Based RL

- Builds an internal model of the environment
- Predicts next state & reward before acting
- More sample-efficient

e.g. Dyna-Q, AlphaZero



Model-Free RL

- No model built — learns from raw experience
- Trial and error directly
- Simpler but needs more data

e.g. Q-Learning, SARSA, PPO



On-Policy

- Learns from actions it is currently taking
- Same policy for exploration & learning
- More stable, slower

e.g. SARSA, PPO, A3C



Off-Policy

- Learns from stored past experiences
- Can reuse old data (replay buffer)
- More flexible & efficient

e.g. Q-Learning, DQN, SAC

Q-Learning navigates environments without a pre-existing map.

	Model-Based RL	Model-Free RL
Knowledge of Environment	Knows Transition (T) and Reward (R) rules in advance.	T and R are completely unknown. The agent operates in the dark.
Core Approach	Computes policy mathematically offline before moving.	Directly computes value function iteratively through online interaction.
Algorithm Example	Dynamic Programming.	Q-Learning (Temporal Difference).

Takeaway: Q-Learning is an off-policy algorithm. It directly approximates the optimal Q-values, $Q^*(s, a)$, independent of the agent's current behavioral policy, even if the agent acts randomly.

Q-Learning

Algorithm

A Complete Walk-Through with the Room Navigation Example

States = Rooms 0–5

Reward R matrix

Q matrix learned

γ (Gamma) = 0.8 • Goal: Reach Room 5 (outside)

What is Q-Learning?



Value-based RL algorithm

Uses Q-values (action-values) to iteratively improve agent behaviour.



Goal

Maximise the Q-value to find the optimal action-selection policy for every state.



Q-Table

Rows = states, Columns = actions.
Stores the 'goodness' of each state-action pair.



Episode

Ends when agent reaches a terminal (goal) state — no further transitions possible.

$$Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \gamma \times \max[Q(\text{next_state}, \text{all_actions})]$$

The Environment: 6 Rooms

6

Rooms (0–4 inside + 5 outside)

5

States for the agent (Rooms 0–4)

100

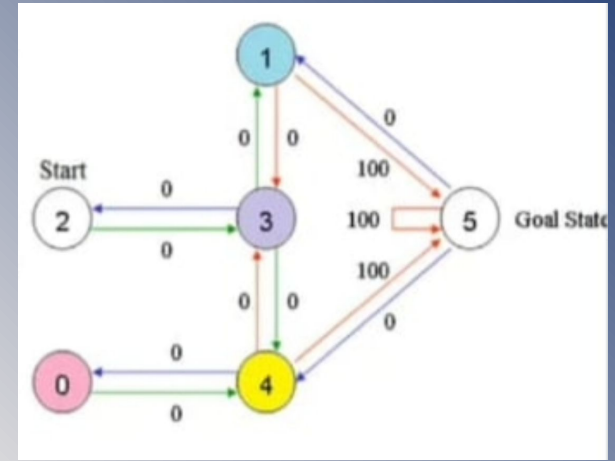
Reward for door leading directly to Room 5

0

Reward for other connected doors

-1

No connection between rooms



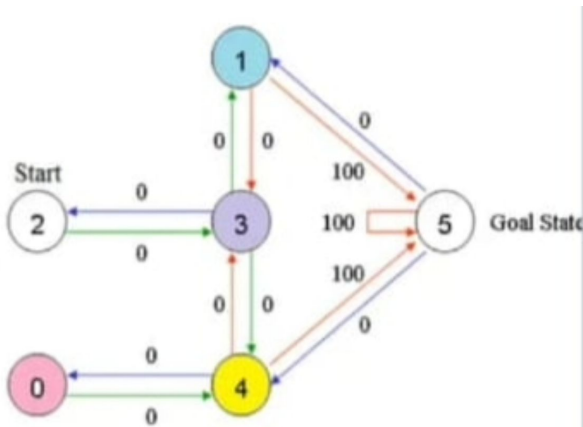
Room 5 = Outside (Goal) — Reward 100 for direct doors to it

Reward Matrix R

-1 No direct connection (door doesn't exist)

0 Connected room, but not directly to goal

100 Door leads directly to Room 5 (outside!)



State \ Action	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

← Rows = current state Columns = action (next room) →

The Q-Matrix (Q)

State 0	0	0	0	0	0	0
State 1	0	0	0	0	0	0
State 2	0	0	0	0	0	0
State 3	0	0	0	0	0	0
State 4	0	0	0	0	0	0
State 5	0	0	0	0	0	0
	Action 0	Action 1	Action 2	Action 3	Action 4	Action 5

The agent begins with zero knowledge of the world.

Learning Rate $\alpha = 1$ |
Discount Factor $\gamma = 0.8$

Q-Learning Algorithm — Step by Step

1

Set γ and init matrices

Set Gamma ($\gamma = 0.8$). Fill Reward matrix R.
Initialise Q matrix with all zeros.

2

Select random start state s

Pick any initial room as current state. Set $s =$ current state.

3

Choose & take an action a

Pick an available action (move to a connected room) using an exploratory policy.

4

Observe reward r , reach s'

Move to next state s' . Look up $R(s, a)$. Compute $\max Q(s', \text{all actions})$.

5

Update $Q(s, a)$

$Q(s, a) = R(s, a) + \gamma \times \max[Q(s', \text{all actions})]$

6

Repeat until goal state

Set $s = s'$. Loop from Step 3 until current state = goal state (Room 5).

Episode 1: Stumbling into the goal state.

1. Observe: Agent starts in State 1.

2. Action: Agent randomly selects Action 5.

3. Execute & Reward: Agent transitions to State 5. Immediate Reward $r = 100$.

4. Update Math:

$$Q(1, 5) = R(1, 5) + \gamma \times \max[Q(5, 1), Q(5, 4), Q(5, 5)]$$

$$Q(1, 5) = 100 + 0.8 \times \max(0, 0, 0)$$

$$Q(1, 5) = 100$$

	Action 0	Action 1	Action 3	Action 4	Action 5
State 0	0	0	0	0	0
State 1	0	0	0	0	100
State 2	0	0	0	0	0
State 3	0	0	0	0	0
State 4	0	0	0	0	0
State 5	0	0	0	0	0

Action 0 Action 1 Action 2 Action 4 Action 5

The agent has recorded its first successful experience.



Episode 1 — Start: Room 2

Path: Room 2 → Room 3 → Room 1 → Room 5 (Goal!)

$$Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \gamma \times \max[Q(\text{next_state}, \text{all_actions})]$$

Step 1 | From Room 2, go to Room 3

$s = 2$ · $a = 3$ · $r = R(2,3) = 0$ · $s' = 3$ · From R: rooms 3 can go to 1, 2, 4

$$\begin{aligned} Q(2,3) &= R(2,3) + 0.8 \times \max[Q(3,1), Q(3,2), Q(3,4)] \\ &= 0 + 0.8 \times \max[0, 0, 0] = 0 + 0 = 0 \end{aligned}$$

$$\rightarrow Q(2,3) = 0$$

Step 2 | From Room 3, go to Room 1

$s = 3$ · $a = 1$ · $r = R(3,1) = 0$ · $s' = 1$ · From R: room 1 can go to 3 or 5

$$\begin{aligned} Q(3,1) &= R(3,1) + 0.8 \times \max[Q(1,3), Q(1,5)] \\ &= 0 + 0.8 \times \max[0, 0] = 0 \end{aligned}$$

$$\rightarrow Q(3,1) = 0$$

Step 3 | From Room 1, go to Room 5 (GOAL!)

$s = 1$ · $a = 5$ · $r = R(1,5) = 100$ · $s' = 5$ (terminal) · From R: room 5 can go to 1, 4, 5

$$\begin{aligned} Q(1,5) &= R(1,5) + 0.8 \times \max[Q(5,1), Q(5,4), Q(5,5)] \\ &= 100 + 0.8 \times \max[0, 0, 0] = 100 + 0 = 100 \end{aligned}$$

$$\rightarrow Q(1,5) = 100 \checkmark$$

Episode 2 — Start: Room 3

Path: Room 3 $\rightarrow$ Room 1 $\rightarrow$ Room 5 (Goal!)

$Q(1,5) = 100$ is now in the Q matrix from Episode 1. This changes the calculation below!

Step 1 | From Room 3, go to Room 1

$s = 3$ · $a = 1$ · $r = R(3,1) = 0$ · $s' = 1$ · Room 1 can go to 3 or 5

$$\begin{aligned} Q(3,1) &= R(3,1) + 0.8 \times \max[Q(1,3), Q(1,5)] \\ &= 0 + 0.8 \times \max[0, 100] = 0 + 0.8 \times 100 = 80 \end{aligned}$$

$$\rightarrow Q(3,1) = 80$$

💡 Now $Q(1,5)=100$ is used! Room 3 gains value because it leads to Room 1 which leads to goal.

Step 2 | From Room 1, go to Room 5 (GOAL!)

$s = 1$ · $a = 5$ · $r = R(1,5) = 100$ · $s' = 5$ (terminal)

$$\begin{aligned} Q(1,5) &= R(1,5) + 0.8 \times \max[Q(5,1), Q(5,4), Q(5,5)] \\ &= 100 + 0.8 \times \max[0,0,0] = 100 \text{ (unchanged)} \end{aligned}$$

$$\rightarrow Q(1,5) = 100$$

💡 $Q(1,5)$ stays 100. Q remains the same because it was already fed into the agent.

Updated Q Matrix after Episodes 1 & 2

Q matrix starts as all zeros. After each episode, non-zero entries appear and grow.

State \ Action	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	100
2	0	0	0	0	0	0
3	0	80	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

From Ep 1
Room 1 → 5 = 100

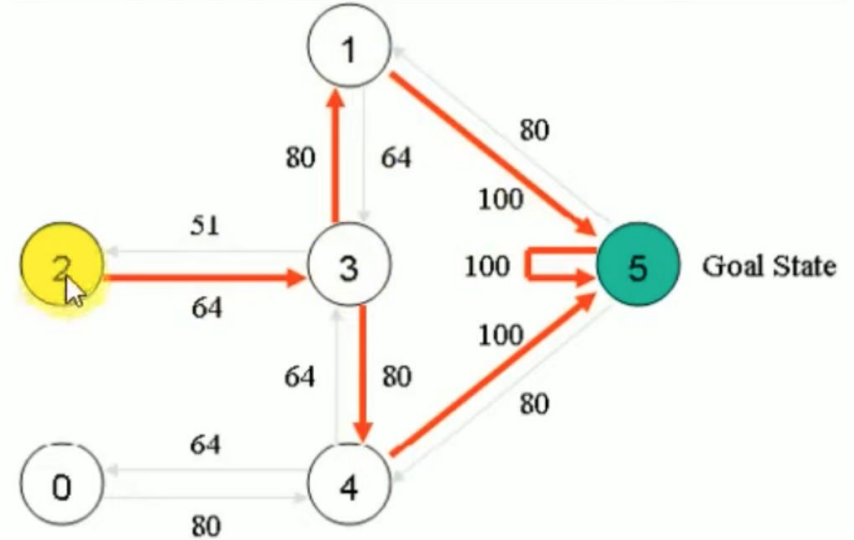
From Ep 2
Room 3 → 1 = 80

After many episodes, more cells fill in → agent learns optimal path from every room to Room 5.

Convergence through repetition

Q values are now converged. We can traverse best possible sequence from the states that maximize long term reward.

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{matrix}$$



Understanding Gamma (γ) — Discount Factor

γ controls how much the agent values FUTURE rewards vs IMMEDIATE rewards. Range: $0 < \gamma < 1$

$$Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \gamma \times \max[Q(\text{next_state}, \text{all_actions})]$$

$\gamma = 0$

Fully short-sighted

Agent only cares about immediate reward R .
Ignores future completely.
Equivalent to greedy policy.

$\gamma = 0.8$

Balanced (used here!)

Future rewards matter, but discounted.
Each step further = multiplied by 0.8 again.
Good balance of now vs later.

$\gamma = 1.0$

Fully far-sighted

Future reward = same as immediate.
Agent plans very long-term.
May not converge in infinite tasks.

With $\gamma=0.8$: $R=100$ at goal $\rightarrow$ 1 step back: $0.8 \times 100 = 80 \rightarrow$ 2 steps back: $0.8 \times 80 = 64 \rightarrow$ 3 steps back: $0.8 \times 64 = 51.2$

Key Takeaways

1

R matrix = environment knowledge

Defines what reward you get for every (state, action) pair. Fixed, set by the designer.

2

Q matrix = agent's learned knowledge

Starts at zero. Updated after every transition. Converges to optimal values.

3

Formula is the engine

$Q(s,a) = R(s,a) + \gamma \times \max_{a'} Q(s', a')$. Reward propagates backward through the Q matrix.

4

Gamma controls future vision

$\gamma = 0.8$ means 2-step future reward is 64% of face value. Balances now vs later.

5

Convergence through repetition

After many episodes starting from random states, Q fills up and stabilises. Then: follow max Q from any room → optimal path to Room 5.